Assignment 5

Gen AI used:

Copilot and Claude. Both accessed on my computer

Prompt:

Write the solutions for each puzzle file to a file named solutionN.txt, where N is the puzzle number.

• Each solutionN.txt file should include the following:

o If no solutions are found, a single line: “No solution found.”

o If solutions are found, it should include all solutions followed by a single line: “Solutions found: n”, where n is the number of solutions found.

o The solutions should be in the same format as the input files: puzzle1.txt – puzzle5.txt.

• You must modify the Assignment 4 you receive, not your Assignment 4.

• Do not replace the entire file with your code. Your GTA will use the Github Blame button to check your modifications.

• Add your name to the Profile Comments as “Revised by:” along with a revision date.

• You must get all required functionality from Assignment 4 working, even if the student you inherited it from did not.

Code:

I listed only the specific edits I was told to make and not the entire code with these edits

Claude:

```
#include <sstream>    // Include string stream library for building solution output text
std::cout << (c == 8 ? "_" : "_ ");   // Print underscore, with a space except at row end
   // Check that the given clues don't already conflict with each other
   bool givensValid() {
```

```
    for (int r = 0; r < 9; ++r) {
        for (int c = 0; c < 9; ++c) {
            if (grid[r][c] != 0) {
                int val = grid[r][c];
                grid[r][c] = 0;                 // Temporarily blank the clue
                bool ok = isValid(r, c, val);     // See if it conflicts with the others
                grid[r][c] = val;                // Restore the clue
                if (!ok) return false;
            }
        }
    }
    return true;
}
    if (givensValid()) {                    // Only search if the clues don't conflict
        solveRecursive(solutionCount, solutions); // Invoke the recursive backtracking solver
    }

    std::ostringstream result;               // Build the output once, send it to terminal and file
    if (solutionCount == 0) {                // Check if the solution count is zero
        result << "No solution found.\n";     // Required message when no solution exists
    } else {
        for (int i = 0; i < solutionCount; ++i) { // Loop through each discovered solution
            int idx = 0;                    // Index tracker for the flattened solution vector
            for (int r = 0; r < 9; ++r) {     // Loop through solution rows
                for (int c = 0; c < 9; ++c) { // Loop through solution columns
                    result << solutions[i][idx++] << (c == 8 ? "" : " "); // Solved cell value
                }
```

```
            result << "\n";              // Newline at end of row
        }
        if (i < solutionCount - 1) {      // Blank line between multiple solutions
            result << "\n";
        }
    }
    result << "\nSolutions found: " << solutionCount << "\n"; // Required summary line
  }

  std::cout << result.str() << "\n";        // Print the same text to the terminal

  std::string outName = "solution" + filename.substr(6); // "puzzleN.txt" -> "solutionN.txt"
  std::ofstream outFile(outName);           // Open the solution file for writing
  if (outFile.is_open()) {                  // Only write if the file opened
    outFile << result.str();              // Write solutions (or the no-solution line)
  }
 }
```

Copilot:

```
void writeSolutionsToFile(const std::string& solutionFilename,
                          const std::vector<std::vector<int>>& solutions,
                          int solutionCount) const {
 std::ofstream outFile(solutionFilename);
 if (!outFile.is_open()) {
  return;
 }
 if (solutionCount == 0) {
  outFile << "No solution found." << std::endl;
```

```
    }
    else {
        for (int i = 0; i < solutionCount; i++) {
            int idx = 0;
            for (int r = 0; r < 9; r++) {
                for (int c = 0; c < 9; c++) {
                    outFile << solutions[i][idx++];
                    if (c < 8)
                        outFile << " ";
                }
                outFile << std::endl;
            }
            if (i < solutionCount - 1)
                outFile << std::endl;
        }
        outFile << "Solutions found: "
                << solutionCount
                << std::endl;
    }
}
std::string solutionFilename = filename;
solutionFilename.replace(6, solutionFilename.size() - 6,
                         filename.substr(6, 1) + std::string(".txt"));
solutionFilename = "solution" + filename.substr(6, 1) + ".txt";
writeSolutionsToFile(solutionFilename,
                     solutions,
                     solutionCount);
```

```
std::string solutionFilename =
    "solution" + filename.substr(6, 1) + ".txt";
writeSolutionsToFile(solutionFilename,
                     solutions,
                     solutionCount);
```

Analysis:

Both Copilot and Claude both run correctly. However, Claudes runs better as it checks clue conflicts and handles invalid inputs. In terms of execution time and space complexity they are both similar. Claude is slightly more efficient with its error checking, however, there is no significant difference. This is because they both use the same recursion and the boards are the same size. They also store each value in similar ways. In terms of maintainability Claude is slightly better as it included more comments explaining the code. Making it easier to understand and modify as necessary.

Selected Program:

I will use Claude's code for my final program. It is the most efficient of the two, requiring less edits to improve it. To improve this code, I would first have it reject files that don't have the 81 cells needed. This improves correctness as it doesn't try to solve a puzzle that is invalid to begin with. I'll also improve the error checking by checking to make sure the files being opened are valid. Finally, I'll improve its solving time by having the program choose an empty cell with the least legal number to solve first. This can allow the program to make less recursive calls as it starts with the most constrained cell which means there's fewer options that are wrong and require backtracking. In terms of maintainability, I'll add more comments explaining the program and use named constants that are easy to change as needed.